ПРОЕКТНОЕ РЕШЕНИЕ

Система автоматизированной доставки контента

Pikabu Bot v3

Репозиторий: github.com/sakurka-cmd/pikabu-bot-v3

Версия документа: 3.1.0

Дата: 09.04.2026

СОДЕРЖАНИЕ

ГЛОССАРИЙ, ТЕРМИНЫ И СОКРАЩЕНИЯ 1ЛИСТ ВЕРСИОННОСТИ 11. ОБЩИЕ ПОЛОЖЕНИЯ 11.1. Бизнес-контекст и обоснование 12. ХАРАКТЕРИСТИКА ОБЪЕКТА АВТОМАТИЗАЦИИ 12.1. Диаграмма бизнес-процесса «To Be» 13. АРХИТЕКТУРНЫЕ РЕШЕНИЯ 13.1. Функционально-компонентная архитектура 14. ТЕХНИЧЕСКИЕ РЕШЕНИЯ 14.1. Подъём и хранение данных 14.2. Базы данных 14.3. Backend-компоненты 14.4. Frontend-компоненты 14.5. Интеграции 14.6. Требования к видам обеспечения 15. ИНФРАСТРУКТУРНЫЕ РЕШЕНИЯ 15.1. Состав технических средств 15.2. Используемые сервисы

Примечание: данное оглавление сформировано с использованием полей Word. Для корректного отображения номеров страниц после редактирования необходимо нажать правой кнопкой мыши на оглавление и выбрать «Обновить поле».

ГЛОССАРИЙ, ТЕРМИНЫ И СОКРАЩЕНИЯ

В настоящем проектном решении используются следующие термины, сокращения и определения, которые необходимы для однозначного толкования текста документа. Понимание данных терминов является обязательным при ознакомлении с архитектурными и техническими решениями, описанными в последующих разделах.

Термин / Сокращение	Определение
VPS	Virtual Private Server — виртуальный выделенный сервер для размещения приложений
API	Application Programming Interface — программный интерфейс приложения для взаимодействия между системами
Docker	Платформа контейнеризации приложений, обеспечивающая изоляцию и переносимость программного окружения
sql.js	Реализация базы данных SQLite на чистом JavaScript, не требующая нативных зависимостей
cheerio	Библиотека парсинга HTML-документов для Node.js, реализующая подмножество jQuery API
Bun	Высокопроизводительный runtime-окружение для JavaScript и TypeScript на базе движка JavaScriptCore
Telegram Bot API	Официальный интерфейс взаимодействия с мессенджером Telegram для создания автоматизированных ботов
windows-1251	Кодировка кириллических символов, используемая на сайте pikabu.ru для представления русского текста
Tag Set	Набор тегов для подписки пользователя (типы: images — только картинки, posts — полные посты)
Spoiler	HTML-тег Telegram <tg-spoiler> для скрытия длинного текста за раскрывающейся подложкой

Таблица 1 — Глоссарий терминов и сокращений

ЛИСТ ВЕРСИОННОСТИ

Ниже представлена история версий проектного решения. Каждая запись содержит номер версии, дату последнего обновления, ответственного исполнителя и краткое описание внесённых изменений. Документ ведётся параллельно с разработкой программного продукта и актуализируется при каждом значительном изменении архитектуры или функциональности системы.

Версия	Дата	Автор	Описание изменений
3.1.0	09.04.2026	Разработчик	Рефакторинг: удалена авторизация, добавлены наборы картинок и постов, поддержка спойлеров
3.0.0	01.04.2026	Разработчик	Начальная версия: парсинг Pikabu, подписки на теги, авторов и сообщества

Таблица 2 — Лист версионности документа

1. ОБЩИЕ ПОЛОЖЕНИЯ

1.1. Бизнес-контекст и обоснование

Сайт pikabu.ru является одной из крупнейших русскоязычных платформ пользовательского контента, ежедневно генерируя тысячи публикаций по широкому спектру тематик. Пользователи, заинтересованные в оперативном получении релевантного контента, сталкиваются с необходимостью многократного ручного просмотра сайта, что требует значительных временных затрат. Задача автоматизации данного процесса является обоснованной с точки зрения повышения пользовательского опыта и экономии времени.

Система Pikabu Bot v3 решает задачу автоматизированного сбора, фильтрации и доставки публикаций с pikabu.ru непосредственно в мессенджер Telegram. Бот предоставляет пользователю гибкий механизм подписок: наборы тегов для получения картинок или полных текстовых постов, подписки на конкретных авторов и сообщества. Функция нечёткого совпадения тегов с поддержкой исключений обеспечивает высокую точность фильтрации контента.

С технической точки зрения система реализована как монолитное однопоточное приложение, развёрнутое в Docker-контейнере на базе Alpine Linux с runtime-окружением Bun. Использование sql.js (SQLite на чистом JavaScript) позволяет избежать нативных зависимостей и обеспечить полную переносимость между платформами. Архитектура системы намеренно минималистична: один контейнер, одна база данных, один процесс — что снижает сложность эксплуатации и повышает надёжность.

Параметр	Значение
Наименование системы	Pikabu Bot v3 — Telegram-бот доставки контента
Назначение системы	Автоматизированный сбор, фильтрация и доставка публикаций с сайта pikabu.ru пользователям Telegram
Заказчик	Внутренний проект (indie-разработка)
Разработчик	sakurka-cmd
Платформа развертывания	VPS Hetzner (Германия, 213.176.117.199), Docker-

Параметр	Значение
	контейнер (Alpine Linux + Bun)
Репозиторий	https://github.com/sakurka-cmd/pikabu-bot-v3
Текущая версия	3.1.0 (09.04.2026)
Язык разработки	TypeScript (ES2022)
Среда выполнения	Bun 1.2
База данных	sql.js (SQLite, in-memory с персистентностью на диск)

Таблица 3 — Основные параметры системы

2. ХАРАКТЕРИСТИКА ОБЪЕКТА АВТОМАТИЗАЦИИ

Объектом автоматизации является процесс сбора, фильтрации и доставки пользовательского контента с платформы pikabu.ru в мессенджер Telegram. До автоматизации данный процесс выполнялся пользователями вручную: периодический визит на сайт, поиск интересных публикаций по тематикам, авторам и сообществам, ручное копирование и пересылка материалов. Это приводило к существенным потерям времени, пропуску релевантного контента и невозможности оперативного реагирования на новые публикации.

Автоматизированный процесс обеспечивает непрерывный мониторинг новых публикаций на pikabu.ru с заданной периодичностью, интеллектуальную фильтрацию по заданным пользователем критериям и мгновенную доставку отобранного контента непосредственно в личные сообщения Telegram. Система поддерживает дедупликацию для исключения повторной доставки одного и того же поста, а также механизм rate limiting для предотвращения блокировки со стороны pikabu.ru.

2.1. Диаграмма бизнес-процесса «To Be»

Ниже представлено описание автоматизированного бизнес-процесса доставки контента в целевом состоянии («To Be»). Процесс состоит из семи последовательных шагов, выполняемых системой автоматически без участия пользователя после первоначальной настройки подписок. Каждый шаг описан с указанием действия, его детального описания и результата выполнения.

Шаг	Действие	Описание	Результат
1	Периодический опрос	Бот выполняет HTTP-запросы к страницам pikabu.ru с заданным интервалом	Сырые HTML-данные
2	Декодирование	Полученный HTML (windows-1251) декодируется	Декодированный HTML

Шаг	Действие	Описание	Результат
		в UTF-8 с использованием полной таблицы кириллицы	
3	Парсинг	Cheerio извлекает заголовки, тексты, изображения, авторов и сообщества из HTML	Структурированные данные постов
4	Фильтрация	Применение правил подписок: совпадение тегов (AND-логика), исключения, проверка наличия изображений	Отфильтрованный список постов
5	Дедупликация	Проверка в таблицах seenPosts и userPosts на предмет повторной доставки	Список новых постов
6	Форматирование	Текст постов обрезается (>300 символов скрываются в <tg-spoiler>), изображения оптимизируются	Готовый контент для отправки
7	Доставка	Отправка сообщений через Telegram Bot API: текст + фото + inline-клавиатуры	Сообщения в Telegram

Таблица 4 — Бизнес-процесс доставки контента (To Be)

После доставки контента система обновляет таблицы дедупликации (seenPosts и userPosts), гарантируя, что один и тот же пост не будет доставлен повторно одному и тому же пользователю. Цикл парсинга повторяется автоматически с интервалом, заданным в настройках бота, обеспечивая непрерывный поток нового контента для всех активных подписок.

3. АРХИТЕКТУРНЫЕ РЕШЕНИЯ

Архитектура Pikabu Bot v3 построена по принципу максимальной простоты и надёжности. Система представляет собой монолитное однопоточное приложение, выполняющее все функции в рамках единого процесса: приём команд от пользователей через Telegram Bot API, периодический парсинг pikabu.ru, фильтрацию контента, взаимодействие с базой данных и доставку результатов. Отсутствие микросервисной декомпозиции обусловлено ограниченной масштабируемостью требований и стремлением к минимальной сложности эксплуатации.

3.1. Функционально-компонентная архитектура

Функционально система разделена на пять логических компонентов, каждый из которых отвечает за определённый аспект работы приложения. Несмотря на физическое нахождение всех компонентов в едином процессе, логическое разделение обеспечивает модульность кода, упрощает тестирование и поддержку. Ниже представлена таблица компонентов с указанием файлов исходного кода и зон ответственности.

Компонент	Файл	Ответственность
Entry Point	src/main.ts	Инициализация базы данных, запуск Telegram-бота, обработка сигналов graceful shutdown (SIGINT, SIGTERM)
Telegram Bot	src/telegram-bot.ts	Обработка команд и callback-запросов пользователей, меню навигации, цикл парсинга, доставка постов, админ-панель
Storage Layer	src/storage.ts	Слой абстракции базы данных: создание таблиц, CRUD-операции для всех сущностей, статистика, автоматическое сохранение каждые 30 секунд
Pikabu Parser	src/pikabu-parser.ts	HTTP-запросы к pikabu.ru, декодирование windows-1251, извлечение данных постов, фильтрация изображений
Credentials	src/pikabu-credentials.ts	Модуль хранения учётных данных (в данный момент не используется после удаления авторизации)

Таблица 5 — Функциональные компоненты системы

Общий объём исходного кода составляет 2125 строк на языке TypeScript. Компоненты взаимодействуют друг с другом напрямую через импорт модулей, без использования промежуточных очередей сообщений или событийных шин. Точка входа (main.ts) выполняет последовательную инициализацию базы данных и Telegram-бота, после чего передаёт управление обработчикам сообщений и таймерам цикла парсинга.

Основные функциональные возможности системы включают: четыре типа подписок (наборы тегов для картинок, наборы тегов для постов, подписки на авторов, подписки на сообщества), нечёткое совпадение тегов по AND-логике с поддержкой исключений, извлечение и фильтрацию изображений из постов, скрытие длинного текста за спойлерами Telegram, механизм дедупликации доставки, администрирование через встроенную панель, а также автоматическое сохранение базы данных каждые 30 секунд.

4. ТЕХНИЧЕСКИЕ РЕШЕНИЯ

4.1. Подъём и хранение данных

Процесс сбора данных с pikabu.ru осуществляется посредством HTTP GET-запросов к публичным страницам сайта. Ответ сервера поступает в кодировке windows-1251, что требует обязательного декодирования в UTF-8 перед дальнейшей обработкой. Для этой цели в компоненте pikabu-parser.ts реализована полная таблица декодирования кириллических символов (всего 128 символов в диапазоне 0x80–0xFF), покрывающая все необходимые символы русского алфавита.

После декодирования HTML-контент анализируется с помощью библиотеки cheerio, которая позволяет извлекать DOM-элементы по CSS-селекторам. Из каждого поста извлекаются: идентификатор (story_id), заголовок, текстовое содержимое, автор, сообщество, список тегов и изображения. Изображения фильтруются по классу story-image, при этом приоритет отдаётся атрибуту data-large-image для обеспечения максимального качества. URL изображений очищаются от параметров трекинга.

Хранение данных организовано с использованием базы данных SQLite через библиотеку sql.js. База данных работает полностью в оперативной памяти (in-memory mode), что обеспечивает высокую скорость операций чтения и записи. Для персистентности данных каждые 30 секунд выполняется автоматическое сохранение содержимого базы на диск в файл ./data/bot.db. При перезапуске приложения база данных автоматически загружается из файла. Данный подход обеспечивает оптимальный баланс между производительностью и надёжностью хранения данных.

4.2. Базы данных

В качестве системы управления базами данных используется sql.js — реализация SQLite, скомпилированная в чистый JavaScript (WebAssembly). Данный выбор обусловлен следующими преимуществами: отсутствие нативных зависимостей (не требуется установка системных пакетов), полная переносимость между платформами, совместимость с Docker-контейнерами на базе Alpine Linux и высокая скорость работы при размещении БД в оперативной памяти.

База данных состоит из девяти таблиц, обеспечивающих хранение всех сущностей системы. Ниже представлена подробная схема базы данных с указанием названий таблиц, столбцов, типов данных и описания каждого поля.

Таблица	Столбец	Тип	Описание
settings	key	TEXT PK	Ключ настройки (например, bot_token, parse_interval)
settings	value	TEXT	Значение настройки
users	id	INTEGER PK	Уникальный идентификатор пользователя (Telegram user_id)

Таблица	Столбец	Тип	Описание
users	username	TEXT	Имя пользователя в Telegram
users	first_name	TEXT	Имя пользователя
users	is_admin	INTEGER	Флаг администратора (0/1)
users	is_blocked	INTEGER	Флаг блокировки (0/1)
users	created_at	TEXT	Дата регистрации
tagSets	id	INTEGER PK	Уникальный идентификатор набора
tagSets	user_id	INTEGER FK	Идентификатор пользователя-владельца
tagSets	name	TEXT	Название набора тегов
tagSets	type	TEXT	Тип: 'images' (только картинки) или 'posts' (полные посты)
tagSets	include_tags	TEXT	Список включаемых тегов (JSON-массив)
tagSets	exclude_tags	TEXT	Список исключаемых тегов (JSON-массив)
tagSets	is_active	INTEGER	Флаг активности набора (0/1)
authorSubscriptions	id	INTEGER PK	Уникальный идентификатор подписки
authorSubscriptions	user_id	INTEGER FK	Идентификатор пользователя
authorSubscriptions	author_name	TEXT	Имя автора на Pikabu
communitySubscriptions	id	INTEGER PK	Уникальный идентификатор подписки
communitySubscriptions	user_id	INTEGER FK	Идентификатор пользователя
communitySubscriptions	community_slug	TEXT	Идентификатор сообщества
seenPosts	post_id	TEXT PK	Идентификатор поста на Pikabu
seenPosts	seen_at	TEXT	Дата первого обнаружения
userPosts	user_id	INTEGER	Идентификатор пользователя

Таблица	Столбец	Тип	Описание
userPosts	post_id	TEXT	Идентификатор поста
dialogStates	user_id	INTEGER PK	Идентификатор пользователя в диалоге
dialogStates	state	TEXT	Текущее состояние конечного автомата
dialogStates	data	TEXT	Временные данные диалога (JSON)
globalStats	key	TEXT PK	Ключ статистики
globalStats	value	INTEGER	Значение счётчика

Таблица 6 — Схема базы данных системы

Все таблицы создаются автоматически при запуске приложения с использованием оператора CREATE TABLE IF NOT EXISTS, что обеспечивает идемпотентность инициализации. Для таблицы settings реализован механизм get/set с автоматическим приведением типов (числа, строки, JSON-объекты). Статистические данные агрегируются в таблице globalStats и обновляются при каждом цикле парсинга и каждой доставке поста пользователю.

4.3. Backend-компоненты

Backend системы реализован целиком на языке TypeScript (стандарт ES2022) и выполняется в runtime-окружении Bun версии 1.2. Выбор Bun обусловлен встроенной поддержкой TypeScript (отсутствие необходимости в отдельном шаге компиляции), высокой производительностью HTTP-запросов и совместимостью с экосистемой npm. Все backend-компоненты работают в едином потоке выполнения, используя асинхронные операции ввода-вывода для неблокирующей обработки.

Компонент	Технология	Описание
Runtime-окружение	Bun 1.2	Высокопроизводительная среда выполнения JavaScript/TypeScript с встроенной поддержкой TypeScript-компиляции
HTTP-клиент	Bun.fetch()	Встроенный HTTP-клиент для выполнения запросов к pikabu.ru с поддержкой кастомных заголовков и cookies
HTML-парсер	cheerio	Серверная библиотека для разбора HTML-документов и извлечения DOM-элементов по CSS-селекторам

Компонент	Технология	Описание
Telegram-клиент	node-telegram-bot-api	Библиотека для взаимодействия с Telegram Bot API, поддержка long polling, отправки сообщений, фото и inline-клавиатур
База данных	sql.js	СУБД SQLite, скомпилированная в WebAssembly/JavaScript, работающая полностью в памяти с возможностью сохранения состояния на диск
Контейнеризация	Docker	Изоляция приложения в Docker-контейнере на базе Alpine Linux для обеспечения воспроизводимости окружения

Таблица 7 — Технологический стек backend-компонентов

Цикл парсинга запускается с интервалом, заданным в настройках бота (по умолчанию — несколько минут). Между HTTP-запросами к pikabu.ru выдерживается задержка в 2–4 секунды для предотвращения блокировки. При получении HTTP-статуса 429 (Too Many Requests) бот приостанавливает парсинг на 60 секунд. Механизм graceful shutdown обеспечивает корректное завершение работы: при получении сигналов SIGINT или SIGTERM приложение сохраняет базу данных на диск и корректно закрывает соединения.

4.4. Frontend-компоненты

В рамках системы Pikabu Bot v3 отсутствует традиционный frontend в виде веб-приложения или мобильного приложения. Роль пользовательского интерфейса выполняет сам бот в мессенджере Telegram, который предоставляет пользователям интерактивное меню для управления подписками и настройки параметров фильтрации.

Пользовательский интерфейс бота организован в виде иерархического меню с использованием inline-клавиатур Telegram. Главное меню содержит следующие разделы:

1. Картинки (🖼️) — управление наборами тегов для получения только изображений из соответствующих постов
2. Посты (📰) — управление наборами тегов для получения полных постов с текстом и изображениями
3. Авторы (👤) — подписка на конкретных авторов Pikabu для получения всех их публикаций
4. Сообщества (👥) — подписка на сообщества Pikabu для получения публикаций из выбранных сообществ
5. Статистика (📊) — просмотр персональной статистики: количество доставленных постов, активные подписки
6. Админ (⚙️) — панель администратора: принудительный запуск парсинга, просмотр общей статистики (доступно только администраторам)

Многошаговые операции (создание и редактирование наборов тегов, добавление подписок) реализованы с использованием конечного автомата (state machine), состояние которого сохраняется в таблице dialogStates базы данных. Это позволяет пользователю прерывать многошаговый процесс и возобновлять его позже без потери введённых данных.

4.5. Интеграции

Система Pikabu Bot v3 интегрируется с двумя внешними сервисами: Telegram Bot API для доставки контента и взаимодействия с пользователями, а также сайтом pikabu.ru для сбора публикаций. Обе интеграции реализованы по модели исходящих соединений — система сама инициирует запросы к внешним сервисам, не предоставляя входящих точек подключения.

Интеграция	Протокол	Направление	Описание
Telegram Bot API	HTTPS (REST)	Исходящий	Отправка сообщений, фотографий и inline-клавиатур пользователям. Приём обновлений через long polling
pikabu.ru	HTTP GET	Исходящий	Получение HTML-страниц с публикациями. Кодировка ответа: windows-1251. Cookies: is_adult=1; adult_mode=1

Таблица 8 — Внешние интеграции системы

Интеграция с Telegram Bot API реализована через библиотеку node-telegram-bot-api с использованием механизма long polling. Бот отправляет сообщения двух типов: текстовые сообщения (с поддержкой HTML-форматирования, включая тег <tg-spoiler> для скрытия длинного текста) и фотографии (с использованием метода sendPhoto). Все сообщения сопровождаются inline-клавиатурами для навигации. Бот также обрабатывает callback-запросы от нажатий кнопок inline-клавиатур.

Интеграция с pikabu.ru реализована через прямые HTTP GET-запросы к публичным страницам сайта. Запросы выполняются с заголовком User-Agent, имитирующим браузер Chrome, а также с заголовком Referer, указывающим на главную страницу pikabu.ru. Для доступа к контенту с пометкой «18+» устанавливаются cookies: is_adult=1 и adult_mode=1. Система implements механизм rate limiting с задержкой 2–4 секунды между запросами и обработкой HTTP-статуса 429 (ожидание 60 секунд при срабатывании ограничителя).

4.6. Требования к видам обеспечения

Для корректного функционирования системы Pikabu Bot v3 необходимо обеспечить следующие виды обеспечения. Каждый вид обеспечения описан с указанием конкретных технологий, инструментов и требований, которые должны быть выполнены для успешного развёртывания и эксплуатации системы.

Вид обеспечения	Описание
Программное	Bun 1.2 (Alpine), TypeScript ES2022, sql.js, cheerio, node-telegram-bot-api. Исходный код — 2125 строк в 5 файлах
Информационное	База данных SQLite (9 таблиц), структура постов pikabu.ru (HTML с windows-1251), Telegram Bot API
Техническое	VPS Hetzner CX22, Docker-контейнер, volume mount для персистентности БД, автоматический перезапуск
Организационное	Ручное управление подписками через Telegram-интерфейс, администрирование через встроенную админ-панель бота
Лингвистическое	Русскоязычный интерфейс бота, поддержка кириллической кодировки windows-1251

Таблица 9 — Требования к видам обеспечения

Системное обеспечение включает Docker Engine и docker-compose для оркестрации контейнера. Базовый образ контейнера — oven/bun:1.2-alpine, что обеспечивает минимальный размер образа и высокую скорость запуска. Политика перезапуска настроена на unless-stopped, гарантируя автоматическое восстановление работы бота после непредвиденных остановок. Docker healthcheck выполняется каждые 30 секунд с использованием команды pgrep bun для проверки жизнеспособности процесса.

Информационное обеспечение включает структуру базы данных из девяти таблиц, описанных в разделе 4.2, а также понимание HTML-структуры страниц pikabu.ru для корректного парсинга. Документация по Telegram Bot API необходима для корректной реализации отправки сообщений, фотографий и inline-клавиатур.

5. ИНФРАСТРУКТУРНЫЕ РЕШЕНИЯ

Инфраструктурное решение для Pikabu Bot v3 основано на принципе минимальной достаточности. Система развёрнута на единственном виртуальном сервере (VPS) провайдера Hetzner, расположенном в дата-центре в Германии. Такой выбор обеспечивает низкую латентность при обращении к европейским серверам Telegram API, а также близкое расположение к серверам pikabu.ru для оптимизации скорости парсинга.

5.1. Состав технических средств

Аппаратная платформа системы включает виртуальный сервер и программные компоненты, обеспечивающие его функционирование. Все компоненты подобраны с учётом требований производительности, надёжности и экономической эффективности. Ниже приведён состав технических средств с детальными характеристиками каждого компонента.

Компонент	Характеристика
Сервер (VPS)	Hetzner Cloud CX22: 2 vCPU, 4 ГБ RAM, 40 ГБ NVMe SSD, Германия (213.176.117.199)
Операционная система	Alpine Linux (внутри Docker-контейнера), хост — Debian/Ubuntu
Контейнерная среда	Docker Engine + docker-compose для оркестрации контейнеров
Среда выполнения	Bun 1.2 — runtime для JavaScript/TypeScript
Хранение данных	SQLite через sql.js (in-memory), персистентность через volume mount: ./data/bot.db
Сетевое взаимодействие	Исходящие HTTPS-соединения к api.telegram.org и pikabu.ru

Таблица 10 — Состав технических средств

Для подключения к серверу используется SSH-доступ с авторизацией по ключу (deploy key). Исходный код доставляется на сервер через Git (clone/pull из репозитория GitHub). Конфигурация развёртывания описана в файлах Dockerfile и docker-compose.yml. Переменная окружения BOT_TOKEN передаётся через файл .env, который исключён из системы контроля версий.

5.2. Используемые сервисы

Для функционирования системы Pikabu Bot v3 используется набор внешних сервисов, каждый из которых выполняет определённую роль в инфраструктуре. Ниже представлена таблица используемых сервисов с указанием провайдера и назначения каждого сервиса.

Сервис	Провайдер	Назначение
VPS-хостинг	Hetzner Cloud	Размещение Docker-контейнера с ботом, обеспечение непрерывной работы (24/7)
Telegram Bot API	Telegram	Доставка контента пользователям, приём команд через long polling
pikabu.ru	Pikabu (публичный сайт)	Источник контента: публикации пользователей, изображения, тексты
Docker Hub	Docker, Inc.	Реестр базовых образов (oven/bun:1.2-alpine) для сборки контейнера
GitHub	GitHub, Inc.	Хранение исходного кода, управление версиями, CI/CD

Таблица 11 — Используемые внешние сервисы

Все внешние сервисы используют стандартные протоколы (HTTPS, SSH, Git) и не требуют установки проприетарного программного обеспечения. Зависимость от pikabu.ru является критической — при недоступности сайта или изменении его HTML-структуры потребуется адаптация компонента парсера. Зависимость от Telegram Bot API также является критической, однако данный сервис отличается высокой степенью доступности (SLA 99.99% согласно официальной документации).

В случае необходимости масштабирования системы возможен переход на более мощный тариф VPS или распределение нагрузки между несколькими серверами. Однако при текущем объёме пользовательской базы монолитная архитектура на одном сервере полностью удовлетворяет требованиям по производительности и доступности.